

Intel e ARM assembly - 05/06/2026

1 Intel x86-64

Esercizio 1

2.75 punti

Quale è una caratteristica distintiva dell'architettura Intel x86-64?

- a) Per usare i registri a 64 bit è necessario utilizzare un prefisso speciale in ogni istruzione (nel nome del registro),
- b) L'architettura supporta solo 8 registri generali a 64 bit,
- c) Non sono presenti istruzioni senza che un registro usato come argomento venga modificato
- d) I flag vengono modificati solo dalle istruzioni "test" e "cmp"

Esercizio 2

2.75 punti

L'indirizzamento basato su registri in Intel x86-64 consente di accedere alla memoria con 4 argomenti, quali sono e come viene calcolato l'indirizzo finale?

- a) Base, Index, Scale, Displacement; Indirizzo = Base + (Index * Scale) + Displacement
- b) Base, Index, Scale, Displacement; Indirizzo = Base + Index + (Scale * Displacement)
- c) Base, Index, Scale, Displacement; Indirizzo = (Base * Scale) + Index + Displacement
- d) Base, Index, Scale, Displacement; Indirizzo = (Base * Index) + (Scale * Displacement)

Esercizio 3

2.75 punti

Riporta quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in Intel x86-64:

```
1 long long int mystery(long long int i, long long int j, long long int f
  , long long int g) {
2     int z = 0;
3     while(z < g){
4         f = f + j;
5         z = z + 1;
6     }
7     return f;
8 }
```

Traduzione Intel x86-64:

```
1 mystery:
2     testq    %rcx, %rcx
3     jle      .L4
4     movl     $0, %edi
5 .L3:
6     movq     %rdi, %r8
7     addq     $1, %rdi
8     cmpq     %rdi, %rcx
9     jne      .L3
```

```

10      addq    %rsi, %rdx
11      imulq   %rsi, %r8
12      X1
13      ret
14 .L4:
15      movq    %rdx, %rax
16      ret

```

Quale delle seguenti istruzioni è corretta per completare la traduzione?

- a) `leaq (%rdx,%rsi), %rax` b) `leaq (%rdx,%rdi), %rax`
- c) `leaq (%rdx,%r8), %rax` d) `leaq (%rdx,%rcx), %rax`
- e) Nessuna delle precedenti

Esercizio 4

2.75 punti

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in Intel x86-64:

```

1 void vectOps(short* a, short* b, short* c, short n){
2     short i=0;
3     while(i<n){
4         if(b[i]>c[i])
5             a[i++] = b[i]-c[i];
6         else
7             a[i++] = c[i]-b[i];
8     }
9 }

```

Traduzione Intel x86-64:

```

1 vectOps:
2     movq    %rdi, %r10
3     movq    %rsi, %r11
4     testw   %cx, %cx
5     jle     .L1
6     movswq   %cx, %rcx
7     movl    $1, %eax
8     jmp     .L5
9 .L6:
10    movq    %rsi, %rax
11 .L5:
12    movzwl   -2(%r11,%rax,2), %r8d
13    movzwl   -2(%rdx,%rax,2), %edi
14    movl     %r8d, %r9d
15    subl     %edi, %r9d
16    movl     %edi, %esi
17    subl     %r8d, %esi
18    cmpw     %di, %r8w
19    cmovg    %r9d, %esi
20    X1
21    leaq     1(%rax), %rsi
22    cmpq     %rcx, %rax

```

```

23         jne      .L6
24 .L1:
25         ret

```

Quale delle seguenti istruzioni è corretta per completare la traduzione?

- a) `movw %si, -2(%rdi,%rax,2)` b) `movw %si, (%r10,%rax,2)`
c) `movw %si, -2(%r10,%rax,2)` d) `movw %r9w, -2(%r10,%rax,2)`

Esercizio 5

2.75 punti

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in Intel x86-64:

```

1 long long int supreamSum(long long int* a, long long int* b, short n,
  short m){
2     short i=0;
3     long long int sum=0;
4     while(i<n && i<m){
5         sum+= a[i]*b[i++];
6     }
7     return sum;
8 }

```

Traduzione Intel x86-64:

```

1 supreamSum:
2     movq    %rdi, %r8
3     cmpw    %dx, %cx
4     cmovg   %edx, %ecx
5     testw   %cx, %cx
6     jle     .L4
7     movswq  %cx, %rcx
8     X1
9     movl    $0, %eax
10    movl    $0, %ecx
11 .L3:
12    movq    (%r8,%rax), %rdx
13    X2
14    addq    %rdx, %rcx
15    addq    $8, %rax
16    cmpq    %rdi, %rax
17    jne     .L3
18 .L1:
19    movq    %rcx, %rax
20    ret
21 .L4:
22    movl    $0, %ecx
23    jmp     .L1

```

Quale delle seguenti opzioni è corretta per completare la traduzione?

- a) X1: `leaq 0(,%rcx,8), %rdi`; X2: `imulq (%rsi,%rax), %rdx`
b) X1: `leaq 0(,%rcx,4), %rdi`; X2: `imull (%rsi,%rax), %edx`

- c) X1: leaq 0(,%rcx,8), %rdi; X2: imulq (%r8,%rax), %rdx
- d) X1: leaq 0(,%rcx,2), %rdi; X2: imulq (%rsi,%rax), %rdx
- e) Nessuna delle precedenti

Esercizio 6

2.75 punti

Nota 1 Esercizio bonus

Il presente esercizio è stato proposto al tutorato svolto in data 06/05/2026, successivamente mi sono reso conto che la difficoltà di questo esercizio è data principalmente da istruzioni Intel x86-64 che non sono state trattate durante il corso e che non sono richieste per la comprensione del corso stesso.

Non avendo trattato queste istruzioni durante il corso, non mi sembra corretto chiedere agli studenti di risolvere questo esercizio, tuttavia, dato che è un esercizio interessante e che può essere risolto con le conoscenze acquisite durante il corso, ho deciso di lasciarlo come esercizio bonus per chi volesse cimentarsi nella sua risoluzione.

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in Intel x86-64:

```

1 void pointers(int* test, int a, int b, int c){
2     int d = 0;
3     while(d<c){
4         if(d % a == 0){
5             d++;
6         }else{
7             d+=2;
8         }
9     }
10    *test = d;
11 }

```

Traduzione Intel x86-64:

```

1 pointers:
2     testl    %ecx, %ecx
3     jle     .L6
4     movl    $0, %r8d
5 .L5:
6     movl    %r8d, %eax
7     cltd
8     idivl   %esi
9     X1
10    X2
11    cmpl    %ecx, %r8d
12    jl      .L5
13 .L2:
14    movl    %r8d, (%rdi)
15    ret
16 .L6:
17    movl    $0, %r8d
18    jmp     .L2

```

Quale delle seguenti opzioni è corretta per completare la traduzione?

- a) X1: `cmpl $1, %edx`; X2: `sbb1 $-2, %r8d`
- b) X1: `cmpl $0, %edx`; X2: `sbb1 $-2, %r8d`
- c) X1: `cmpl $1, %edx`; X2: `adc1 $2, %r8d`
- d) X1: `testl %edx, %edx`; X2: `sbb1 $-1, %r8d`

2 ARM

Esercizio 7

2.75 punti

Quale è una caratteristica distintiva dell'indirizzamento in ARM?

- a) In ARM, l'indirizzamento basato su registri consente di accedere alla memoria utilizzando solo 2 argomenti: Base e Displacement, con l'indirizzo calcolato come $\text{Base} + \text{Displacement}$.
- b) In ARM, l'indirizzamento basato su registri consente di accedere alla memoria utilizzando 2 o 3 argomenti: Base e Index, con l'indirizzo calcolato come $\text{Base} + \text{Index} * \text{Scale}$, oppure Base, Offset, con l'indirizzo calcolato come $\text{Base} + \text{Offset}$.
- c) In ARM, l'indirizzamento basato su registri consente di accedere alla memoria utilizzando 3 argomenti: Base, Index e Scale, con l'indirizzo calcolato come $\text{Base} + \text{Index} * \text{Scale}$.
- d) In ARM, l'indirizzamento basato su registri consente di accedere alla memoria utilizzando 3/4 argomenti: Base, Index, Scale e Displacement, con l'indirizzo calcolato come $\text{Base} + (\text{Index} * \text{Scale}) + \text{Displacement}$.

Esercizio 8

2.75 punti

L'istruzione di caricamento multiplo in ARM consente di caricare più registri da memoria in un'unica istruzione. Quale è la sintassi corretta per questa istruzione e come viene specificato l'elenco dei registri da caricare?

- a) L'istruzione di caricamento multiplo in ARM utilizza la sintassi: `LDM{cond} Rn!, {Rlist}`, dove Rn è il registro di base, Rlist è una lista di registri racchiusa tra parentesi graffe {} che specifica i registri da caricare, e il punto esclamativo (!) indica che il registro di base viene aggiornato dopo il caricamento.
- b) L'istruzione di caricamento multiplo in ARM utilizza la sintassi: `LDM{cond} Rn, {Rlist}`, dove Rn è il registro di base, Rlist è una lista di registri racchiusa tra parentesi graffe {} che specifica i registri da caricare, e il punto esclamativo (!) indica che il registro di base non viene aggiornato dopo il caricamento.
- c) L'istruzione di caricamento multiplo in ARM utilizza la sintassi: `LDM{cond} Rn!, {Rlist}`, dove Rn è il registro di base, Rlist è una lista di registri racchiusa tra parentesi graffe {} che specifica i registri da caricare, e il punto esclamativo (!) indica che il registro di base non viene aggiornato dopo il caricamento.
- d) L'istruzione di caricamento multiplo in ARM utilizza la sintassi: `LDM{cond} Rn, {Rlist}`, dove Rn è il registro di base, Rlist è una lista di registri racchiusa tra parentesi graffe {} che specifica i registri da caricare, e il punto esclamativo (!) indica che il registro di base viene aggiornato dopo il caricamento.

Esercizio 9

2.75 punti

Esiste una procedura generica “multAccess” che esegue delle operazioni di lettura e scrittura su più vettori di interi, questa a sua volta per semplificare i calcoli al suo interno invoca una tal funzione “calc” che a partire dal vettore in origine, la dimensione del vettore e un indice restituisce un intero, questa funzione è la seguente

```
1 int calc(int a[], int i, int n) {
2     return a[i] + a[n/2+i];
3 }
```

Considerando che la funzione “multAccess” invoca la funzione “calc” rispettando le convenzioni di chiamata, quali sono le istruzioni in ARM mancanti nella seguente porzione di codice che rappresenta la funzione “multAccess”?

```
1 calc:
2     add    r2, r2, r2, lsr #31
3     add    r2, r1, r2, asr #1
4     ldr    r2, [r0, r2, lsl #2]
5     ...
6     add    r0, r0, r2
7     bx     lr
```

- a) ldr r0, [r2, r1, lsl #2]
- b) ldr r1, [r2, r1, lsl #2]
- c) ldr r3, [r0, r1, lsl #2]
- d) ldr r0, [r2, r1, lsl #2]
- e) Nessuna delle precedenti

Esercizio 10

2.75 punti

Viene data la seguente funzione in C:

```
1 int mystery(int a[], int n, int b[], int m){
2     int res=0;
3     for(int i=0;i<n && (res<a[i]||res<b[i]) && i < m;i++){
4         res += a[i] + b[i/2];
5         a[i] = b[i];
6     }
7     return res;
8 }
```

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in ARM:

```
1 mystery:
2     push   {r4, r5, r6, r7, lr}
3     subs   r5, r1, #0
4     ble    .L6
5     mov    r6, r2
6     mov    r7, r3
7     sub    lr, r0, #4
8     subs   r2, r2, #4
9     mov    ip, #0
10    mov    r0, ip
11    b      .L3
```

```

12 .L4:
13     X1
14     add    r3, ip, ip, lsr #31
15     asrs   r3, r3, #1
16     ldr    r3, [r6, r3, lsl #2]
17     add    r1, r1, r3
18     add    r0, r0, r1
19     ldr    r1, [r2, #4]!
20     X2
21     add    ip, ip, #1
22     cmp    r5, ip
23     beq    .L1
24 .L3:
25     add    lr, lr, #4
26     mov    r4, lr
27     ldr    r1, [lr]
28     cmp    r1, r0
29     bgt    .L4
30     ldr    r3, [r2, #4]
31     cmp    r3, r0
32     bgt    .L4
33 .L1:
34     pop    {r4, r5, r6, r7, pc}
35 .L6:
36     movs   r0, #0
37     b      .L1

```

- a) X1: cmp r7, ip; bge .L1 X2: str r0, [r4]
- b) X1: cmp r7, ip; ble .L1 X2: str r1, [r6]
- c) X1: cmp r7, ip; ble .L1 X2: str r1, [r4]
- d) X1: cmp r7, r0; blt .L1 X2: str r1, [r2]
- e) Nessuna delle precedenti

Esercizio 11

2.75 punti

Considerando la seguente funzione in C:

```

1 void mystery(int a[], int n, int b[], int m, int* c){
2     for(short i=0;i<a[i]&& i<b[i];i++){
3         if(a[i]>b[i])
4             *(c++) = *(a++) - *(b++);
5         else
6             *(c++) = *(b++) - *(a++);
7     }
8 }

```

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in ARM:

```

1 mystery:
2     ldr    r1, [r0]
3     cmp    r1, #0

```

```

4      ble      .L8
5      push     {r4, r5, r6, lr}
6      add      lr, r0, #4
7      ldr      r4, [sp, #16]
8      adds     r4, r4, #4
9      movs     r3, #0
10     mov      r0, r3
11     mov      ip, r3
12 .L3:
13     mov      r6, r2
14     ldr      r0, [r2, r0]
15     X1
16     ble      .L1
17     cmp      r0, r1
18     ittee     lt
19     ldrlt    r1, [lr, #-4]
20     ldrlt    r0, [r2]
21     ldrge    r1, [r6]
22     ldrge    r0, [lr, #-4]
23     X2
24     str      r1, [r4, #-4]
25     adds     r3, r3, #1
26     sxth     r3, r3
27     mov      ip, r3
28     lsls     r0, r3, #2
29     ldr      r1, [lr, r3, lsl #2]
30     adds     r2, r2, #4
31     add      lr, lr, #4
32     adds     r4, r4, #4
33     cmp      r3, r1
34     blt      .L3
35 .L1:
36     pop      {r4, r5, r6, pc}
37 .L8:
38     bx       lr

```

Quale delle seguenti opzioni è corretta per le etichette X1 e X2?

- a) X1: `cmp r3, r1`; X2: `sub r1, r1, r0`
- b) X1: `cmp r0, r3`; X2: `add r1, r1, r0`
- c) X1: `cmp r0, r3`; X2: `sub r1, r1, r0`
- d) X1: `cmp r0, r1`; X2: `sub r0, r0, r1`
- e) Nessuna delle precedenti

Esercizio 12

2.75 punti

Viene fornita la seguente funzione in C:

```

1 void copia_stringa(char d[], char s[]) {
2     int i = 0;
3     while (1) {
4         char c = s[i];

```



```

5         d[i] = c;
6         if (c == '\0') break;
7         i++;
8     }
9 }

```

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in ARM:

```

1 copia_stringa:
2     ldrb    r3, [r1]
3     strb    r3, [r0]
4     cbz     r3, .L1
5 .L3:
6     X1
7     cmp     r3, #0
8     bne     .L3
9 .L1:
10    bx      lr

```

Quale delle seguenti opzioni è corretta per le etichette X1 e X2?

- a) ldrb r3, [r1], #1 ; strb r3, [r0], #1
- b) ldrb r3, [r1, #1]! ; strb r3, [r0, #1]!
- c) ldrb r3, [r1, r2] ; strb r3, [r0, r2]
- d) ldrb r3, [r1, #4]! ; strb r3, [r0, #4]!
- e) Nessuna delle precedenti